

Parametrized and Exact Algorithms

Notes by Arjun Maneesh Agarwal

based on course by G. Philip

The main book will be Parametrized Algorithms by Cygan et. al (chapters 1 to 7, 13 and 15). Other than that < copy from moodle >.

Table of Contents

1. Philip's Wisdom	1
2. Brief Intro and Motivation	1

1. Philip's Wisdom

"Why do you want to solve an algorithms problem, probably to make more money. Or kill more people. Or maybe save more people."

2. Brief Intro and Motivation

Parametrized Algorithms starts where the first algorithms course ends: NP Completeness.

Most algorithms we saw in the algorithms course have complexity $O(n^c)$ for some constant c . This raises the question if that is true of all problems, sadly, this is where NP Hardness and Completeness comes in. While we don't know if we can solve them polytime, these NP Complete problems span thousands of fields and experts in their field have tried finding fast algorithms. As these problems are complete, all reduce to each other which implies that if one of them succeed, all of them succeed. Unfortunately, they have all failed, which is the sort of silence that speaks quite loudly.

So how do we cope with this? One is Approx Algorithms. The other idea is Random Algorithms. Another is heuristics and meta-heuristics (taken from physics and biology and what not).

In theory, NP Completeness implies all the problems are equally hard modulo some polynomial time translations. However, some problems were easier to solve in practice.

Example

Type Inference for Hindley-Milner type system is unconditionally not solvable in polynomial time (in the size n of the input programme) as it hard for the class EXP (that is problems that can be solved in $O(2^{\text{poly}})$ time) (from Maivson, POPL 1990) and $P \subsetneq \text{EXP}$ (Time Hierarchy theorem)

However, we don't think about the size of programme while writing code. And the compiler happily (or sadly) infers the type quite quickly.

Theorem 2.1. *Type inference for Hindley-Milner Type system can be solved in $O(2^k n)$ time where*

- *n is the size of input programme,*
- *k is the nesting depth of types in the programmes*

(Lichtenstein and Porveli, POPL 1985)

The idea is to pick something that tends to something that is usually small in practice and limit the exponential blow up to that parameter.

So our roadmap is: pick a parameter k that could be natural or something more convoluted, try to find an algorithm of time complexity $O(f(k)n^c)$. If we succeed the problem is Fixed Parameter Tractable aka FPT.

Definition: Vertex Cover

Vertex cover of a graph G is a subset $S \subseteq V(G)$ such that $G \setminus S$ has no edges.

Definition: Vertex Cover (Standard Parametrization)

Input: Graph G , $k \in \mathbb{N}$

Parameter: k

Question: Does G have a vertex cover of size $\leq k$?

Remark

This can be thought of as choosing a subset as guard posts such that all the roads are guarded.

Also, please make sure to not confuse this with **Dominating Set** (which is choosing vertices to put 1 edge radius umbrellas to cover everyone).

While there are ways to